Imperial College
London

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Programming Model for a Stream Graph Processing System

Author:
Luca Campese

Supervisor:
Prof Peter Pietzuch

Second marker:
Dr Jana Giceva

Submitted in partial fulfillment of the requirements for the MSc degree in Advanced Computing of Imperial College London

September 2018

Abstract

Processing dynamically changing streaming graph data is an exciting new challenge in the field of large-scale data processing nowadays. Kappa is a system which tackles such computations, although it did not provide a programming interface for a user to express a graph algorithm of interest. In my thesis, I propose a programming model for this system which lets a user define a computation on a dynamically changing graph in a simple and intuitive way. By implementing three algorithms of different type using this programming model, I demonstrate the flexibility and generality of my approach.

Acknowledgments

I would like to first thank my supervisor Prof Peter Pietzuch and my second marker Dr Jana Giceva for their guidance and patience throughout this project. Secondly, I would like to express my gratitude to Domagoj Margan for the many hours spent together working on Kappa, and to all the friendly people I have met in the LSDS research group. Lastly, I would also like to thank my friends and family for their support during my studies. A special thank you goes to Eduardo for his continuous encouragement throughout this year.

Contents

Chapter 1

Introduction

Efficient processing of graph data has been subject of research within the last decade. Since the development of Pregel [9], many graph processing systems have emerged to target different environments: broadly speaking, they can be categorised into single-machine and distributed systems. Depending on their objectives, certain systems use main memory to achieve faster performance, while others take advantage of secondary storage to support processing of very large amounts of data.

Although, the vast majority of existing graph processing systems focuses on static graphs only. Thus, they are not able to handle dynamic graph-structured data, be it in form of temporal or streaming data, which is commonly generated in several real-world scenarios. One can think of a highly dynamic network such as a social network: as an example, 500 million tweets are created every day on Twitter [14].

Running computations efficiently on such dynamically evolving graphs is the main goal of Kappa, a stream graph processing system being actively developed in the LSDS research group at Imperial College London.

In this report, I present my contributions to Kappa, which revolve mainly around the programming model I designed to let a user express an algorithm on a dynamic graph. The structure of the report is as follows. Chapter 2 describes the design principles behind the most representative graph processing systems, which are used to position Kappa in the field of graph processing. Chapter 3 presents the design of the system, whereas in Chapter 4 the discussion points to the implementations of some well-known graph algorithms in Kappa. Chapter 5 contains a description of the experiments and the environment on which they have been carried out. Finally, in Chapter 6 I suggest possible future directions for Kappa and conclude this report.

Chapter 2

Background

When building a graph processing system, there are several aspects one needs to take into consideration. In this chapter, I describe the rationale behind certain design ideas which serves as background to understand where Kappa stands compared to other systems. Furthermore, related work is discussed at the end of the chapter.

The following sections (except the last one) have been extracted from my ISO project, whose aim was to abstract away the design principles underpinning the most representative graph processing systems presented in the literature, and should therefore not be considered for grading.

2.1 Programming model

Different graph processing engines let the user express the algorithm he or she wants to execute in different ways. If the algorithm is called on every vertex, the computation is said to be **vertex-centric**; on the contrary, the computation is **edge-centric** if the algorithm is executed on each edge.

In this section, these two paradigms will be explored in more detail.

2.1.1 Vertex-centric computations

In a **vertex-centric** programming model, the user supplies a function which is executed on all (or some of) the vertices of the input graph—for this reason, the programmer is encouraged to *think like a vertex*. This model was first presented in Pregel [9], a distributed graph processing system developed at Google.

In Pregel, a user-defined function is supplied by subclassing the `Vertex` class—provided by the framework—and overriding its `Compute()` method. By doing this, the user can access the values associated with a vertex and its edges, and can also have a vertex communicate with other vertices by sending messages to them, as described in Section 2.3.1.

Listing 2.1: The `Vertex` class provided by Pregel [9].

```
template <typename VertexValue,
          typename EdgeValue,
          typename MessageValue>
class Vertex {
  public:
    virtual void Compute(MessageIterator* msgs) = 0;

    const string& vertex_id() const;
    int64 superstep() const;

    const VertexValue& GetValue();
    VertexValue* MutableValue();
    OutEdgeIterator GetOutEdgeIterator();

    void SendMessageTo(const string& dest_vertex,
                       const MessageValue& message);
    void VoteToHalt();
};
```

2.1.2 Edge-centric computations

In an **edge-centric** programming model, instead of applying a user-defined function on every vertex, each iteration of a computation proceeds edge by edge. X-Stream [12] introduced this approach based on the fact that sequential access to storage is faster than random access due to hardware prefetching, be it from main memory to CPU cache or from disk to main memory: instead of sorting and indexing the list of edges, this system streams them from storage and chooses the ones linked to the active vertices during every iteration. X-Stream divides the computation into two phases: the first one—referred to as *scatter* phase—reads the outgoing edges from the input stream (and the relative source vertices which are pinned in memory), and might produce updates written to an output stream sequentially; the second one—referred to as *gather* phase—reads all the updates along the incoming edges generated during the previous phase and modifies ac-

cordingly the values stored in the target vertices. It should be noted that both operations do not make any assumption on the order the edges and updates are streamed in.

Listing 2.2: The edge-centric computation model proposed by X-Stream [12].

```
edge_scatter(edge e)
  send update over e

update_gather(update u)
  apply update u to u.destination

while not done
  for all edges e
    edge_scatter(e)
  for all updates u
    update_gather(u)
```

This model is well suited for graphs whose edge set is significantly larger than its vertex set. In case the latter does not not fit in memory, the graph is split into multiple partitions. Then, the computation will iterate through the partitions rather then through the edges.

To show how this model differs from the vertex-centric one described above, one can think of how PageRank—described in Section 4.1—can be expressed as an edge-centric computation: during the scatter phase, for each streamed edge *e* an update is produced by calculating $value(v)/degree_{out}(v)$ where v is the in-memory source vertex of e. Then, during the gather phase each update u is used to increase $value(v)$ by $0.85 \cdot u$. The computation proceeds to the next iteration when this latter phase finishes.

2.2 Information flow model

In different graph processing systems, vertices **push** or **pull** information to or from their neighbours. This section describes how the two approaches differ and when it is appropriate to combine them.

2.2.1 Pushing data

When **pushing** data, vertices send information across their outgoing links, and can potentially write their neighbours' states. For this reason, locks are often used to handle concurrent writes to the same vertex. On the other hand, one can save computation power by running the algorithm on just a subset of the graph: in fact, vertices that will not push any data during a certain iteration can be discarded for that iteration. This is not possible in **pull** mode, as explained below.

2.2.2 Pulling data

When **pulling** data, each vertex checks whether there is some value to be read on any of its incoming edges. The edges of each vertex do not need to be *all* scanned at every iteration: for instance, if a vertex sees it has already been discovered during BFS through one of its edges, it can avoid looking at its other edges. Conversely, in push mode several vertices might push data to the same destination vertex d during a certain iteration in BFS, since they have no way to know that d has been indeed discovered by multiple vertices. In such an iteration, much redundant work would potentially be done.

With regard to synchronisation, locks can be avoided in this mode since nodes update only their own state.

2.2.3 Push-pull mode

As observed above, pushing and pulling data have different advantages and disadvantages. Therefore, it is possible to adopt a hybrid approach known as **push-pull** mode. This comes at the cost of extra pre-processing needed to build the adjacency lists for both the outgoing and incoming edges.

Ligra [13] switches between the two modes depending on the number of active vertices at any given iteration: when this number is small, the outgoing edges of the active vertices are processed by checking whether each destination vertex has to be updated. Intuitively, this check may be performed multiple times when the number of active vertices (plus their outgoing edges) is large. In such a case, Ligra switches from **push** to **pull** mode and processes the incoming edges (s, d) where s is an active vertex and d is a vertex that is to be updated: checking that d is to be updated is performed only once. To convey this idea more concretely, one can use BFS again as an example: in Ligra, the update function simply writes the parent of each vertex v in `parent[v]` where `parent` is an array. A vertex needs to

be updated if its parent has not been written yet. Now, let us assume that a vertex d has already been discovered, and that all its in-neighbours (but one) are active: then, d would be checked $degree_{in}(d) - 1$ times, instead of just one time in push mode. As the number of active vertices increases, this behaviour is more likely to happen, and this is why Ligra would switch to pull mode.

2.3 Communication model

When designing a graph processing system, one needs to decide how vertices exchange data. The two most common abstractions exposed by graph processing systems are **message passing** and **shared memory**.

2.3.1 Message passing

Message passing is a mechanism by which a process can invoke another process' behaviour by sending it a message. In a graph processing system, opting for a communication model based on message passing means having vertices communicate with each other by sending messages, and expecting the receiving vertices to act upon those.

In Pregel [9], a vertex can send any number of messages to any other vertex, provided that the destination vertex ID is known by the sender. This latter aspect is particularly useful when two vertices far apart from each other want to communicate: in fact, this allows for fast value propagation which would not be possible in other models where vertices can send messages to their neighbours (or access their data) only. In the literature, the algorithms exhibiting this behaviour are referred to as *pointer-jumping* algorithms. A per-vertex queue stores all the incoming messages for an iteration, so that they will be available during the subsequent one.

Any distributed graph processing system will have to deal with the cost of network communication caused by sending messages to vertices stored on a remote machine. This cost can be minimised by placing related vertices on the same machine: similarly to the approach taken to store web pages in Bigtable [1]—a distributed storage system also developed at Google—nodes that identify pages of the same site should be co-located. Furthermore, Pregel offers *combiners* to reduce the number of messages sent to a vertex during an iteration to only one. For example, when running an SSSP algorithm, a vertex is interested in receiving only the message containing the minimum distance during a given iteration, as the messages with larger distances will be discarded: a combiner would implement this logic. While this greatly reduces message traffic in such an algorithm, this

behaviour needs to be coded by the user, since it is dependent on the semantics of the algorithm.

Another idea to decrease network communication, introduced and implemented in a system named Pregel+ [15], is to replicate some vertices across multiple machines. The rationale behind it is based on the fact that a high-degree vertex v might send a potentially high number of messages: therefore, by replicating v onto all the machines containing part of its neighbours, the number of inter-machine messages sent by v cannot be greater than the total number of machines, with the latter often being much smaller than $degree_{out}(v)$. In fact, the copies of v can act as proxies by delivering a message to v's *local* neighbours without incurring in network-related costs.

2.3.2 Shared memory

Systems that use a **shared-memory** abstraction differ from those that use message passing in that a vertex v can access its neighbours' data directly. Contrary to message passing, this model does not support reading values of non-neighbouring vertices. This means that pointer-jumping algorithms are not supported, making this model less expressive.

Distributed GraphLab [8] makes use of this model motivated by the nature of machine learning algorithms, in which certain vertices converge faster than others. A concrete example is given in [8] to describe how this model works: in PageRank, a vertex v would read its neighbours' values to update $value(v)$, and then decide if its neighbours are to be scheduled for execution based on whether $value(v)$ has changed sufficiently[1]. This system does not expose the complexity of network communication at the cost of employing locks to protect against updates to the same memory locations: despite this, the results of the execution of algorithms such as PageRank are not deterministic. In addition, a pair of vertices belonging to two different machines need to be replicated and synchronised on both machines. This becomes an issue with regard to scalability: indeed, the input graphs used to evaluate this system in [8] are considerably smaller than the ones used in Pregel [9], for example.

[1] $|value(v) - value_{old}(v)| > \epsilon$ where ϵ is a predefined threshold value.

2.4 Execution model

In most graph processing systems, execution proceeds in iterations: values stored in the graph are updated until the execution terminates. If the transition from one iteration to the next one is preceded by synchronisation points, the execution is **synchronous**. On the contrary, when this aspect is relaxed, the execution is **asynchronous**.

2.4.1 Synchronous execution

This execution model was first described in Pregel [9], in which iterations are referred to as *supersteps*. During any superstep, a vertex can only access the incoming messages sent during the previous superstep. This makes execution more predictable and easy to reason about, since its result does not depend on the order in which messages are sent. The source of inspiration for this model was the **bulk synchronous parallel** (**BSP**) model, which encapsulates the idea of local computations waiting for each other before proceeding to the next computation step.

Similarly, X-Stream [12] guarantees that any gather phase has access to all the updates sent during the previous (completed) scatter phase. The scatter-gather model is covered in Section 2.1.2.

From an efficiency point of view, requiring synchronisation between iterations might seem to introduce significant overheads. However, this is mitigated by sending messages in batches and by the fact that an algorithm is guaranteed to terminate with the same result across multiple executions. On the downside, synchronicity poses the problem of dealing with stragglers—namely, one slow worker machine might have a negative impact on the whole system. For example, Pregel does not have a way to deal with this issue.

2.4.2 Asynchronous execution

In this execution model, a vertex can see the messages (resp. the updates) sent (resp. applied) to itself during an iteration without having to wait until the next one. In this model, concurrent accesses can indeed happen: these are typically handled by locking and unlocking resources—namely, vertex values. Furthermore, results given by asynchronous executions may vary or not be exactly accurate due to race conditions emerging during the updates.

Having said that, this model leads to faster execution times for the algorithms that can tolerate such inaccuracies: in PageRank, vertices converging at a faster rate do not have to wait for vertices taking longer to finish.

GraphLab [8] exposes this model of execution by utilising a scheduler that processes the vertices asynchronously: the main difference with Pregel-like systems is that the computation does not follow bulk-synchronous steps. This means that user-defined vertex functions can be executed at any time, provided that the set consistency model is not violated. In practice, the degree of parallelism decreases under stronger consistency guarantees.

The three models offered by GraphLab can be seen in Figure 2.1: they are satisfied by first using heuristics to colour the graph, and then by executing in parallel the vertices sharing the same colour. *Edge consistency*, for instance, is guaranteed by assigning a colour c to each vertex such that none of its neighbours is given c, too.

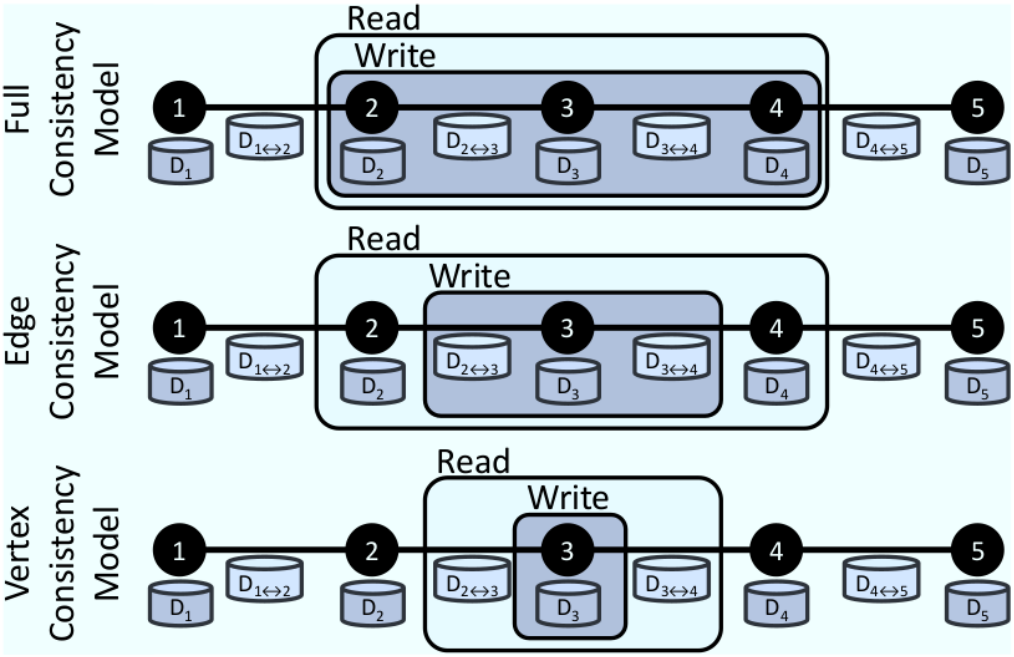

Figure 2.1: Different consistency models offered by GraphLab [8].

2.5 Platform model

Another factor that influences the design of a graph processing system is the platform it targets. Not only certain systems are designed to run on a single machine

whether others are distributed, but they vary on the hardware required to obtain optimal performance.

Distributed graph processing systems tend to keep the input graph **in memory**, since a cluster of commodity computers can provide a large amount of RAM. This is done to alleviate the cost of network communication between machines.

On the other hand, **out-of-core** systems use secondary storage to process large graphs, typically on a single machine. In this case, a great deal of attention is given to disk I/O as this is the factor that bounds the speed of such systems.

GraphChi [5] was one of the first systems to be able to process large graphs on a commodity PC. It exposes an asynchronous model in which the vertices are range-partitioned and sharded together with their in-edges. Then, the computation proceeds shard by shard, storing in main memory the content of one shard and loading the out-edges from secondary storage: this method is referred to as *Parallel Sliding Windows*. Since the content of the shards is sorted, loading the out-edges from disk can be thought of as sliding a window over each of them. This allows for sequential reads rather than random accesses to locate the out-edges, which is what GraphChi exploits.

X-Stream [12], whose model of computation is described in Section 2.1.2, is another system which offers both in-memory and out-of-core executions. Roy et al. report in [12] how their system outperforms GraphChi even for out-of-core execution when pre-processing time is taken into account (and when using a SSD to read data from). In fact, GraphChi needs to sort the edges of the input graph, whereas X-Stream does not need to. Furthermore, when the vertex set does not fit in memory, X-Stream creates a smaller number of partitions.

Interestingly enough, an out-of-core system does not necessarily have to run on a single machine only. Chaos [11] builds on top of X-Stream to scale out its capabilities by fully utilising the cumulative disk space of a small cluster, which is treated as flat storage.

2.6 Load balancing

Graph algorithms tend to show a varying degree of parallelism during execution. For example, in Hash-Min—an algorithm which propagates the smallest value of a vertex to all the vertices in the same component—just a few nodes are active after the first iteration.

This characteristic might lead to having machines with significantly different workloads. Nevertheless, it is quite difficult to address this issue at run time. In fact,

moving vertices between machines according to how heavily loaded each machine is would invalide cheap mapping strategies such as hashing a vertex ID in order to locate it. In addition, a vertex might happen to finish its work just after being moved to another machine, thus making the operation of moving it ineffective. For this reason, **dynamic repartitioning** has not gained much popularity so far.

Work stealing is another technique to achieve load balancing at run time: Chaos [11], for instance, implements this by having workers which are finished processing their partition help other workers which have still processing to do. During the scatter phase, this means that a helper machine would need to copy into its memory the vertex set of a certain partition, and then process the chunks of the edges (which have not been processed yet) belonging to the same partition. The gather phase is more subtle, as in this case some vertices live on different machines and would need to be kept in sync. The approach taken by Chaos is to compute the final value of a replicated vertex by combining together the partial values stored on different machines using a user-defined `apply` function at the end of this phase.

2.7 Related work

As mentioned in Chapter 1, real-world graphs evolve and change over time—thus, static graph processing systems are not suitable for efficient processing of such data. As a response to this limitation, several publications presented systems designed for processing of dynamically changing graph data.

Microsoft Research's **Kineograph** [2] was the first system for such purpose: in short, it is a distributed message-passing system that supports vertex-centric scatter-gather computations over streams of graph data. Kineograph builds consistent snapshots of the input graph out of accumulated graph updates. Then, this system processes the snapshots incrementally by reusing previously computed vertex states coming from previous snapshots as initial values for the current one. Conflicts between graph updates and computations are avoided by fully separating these two processes: computation is carried out on existing snapshots while new updates are used to create new snapshots. The authors report that Kineograph was deployed on a cluster of up to 50 machines and was capable of processing a Twitter-based graph stream which reached 8 million vertices and 29 million edges at its peak.

Chronos [3] was introduced as follow-up work on Kineograph. This system processes collected temporal graph data by incrementally computing results over a series of static snapshots within a fixed time range, loaded from disk storage rather than from the network. Chronos can be deployed onto a single or multiple ma-

chines. Similarly to Kineograph, Chronos computes only over graph snapshots, thus avoiding issues related to data dependency and conflicts between temporal changes and computations happening at the same time in the graph.

Naiad [10] has demonstrated its ability to successfully process dynamic graphs, even though it supports many types of data rather than specialising on graphs only. Naiad is a general-purpose distributed dataflow system designed for computing over continuously changing inputs of data. It offers support for a variety of high-level programming models which can be built on top of its dataflow engine, including the vertex-centric model. In contrast to Kineograph and Chronos, whose incremental processing requires preserving temporal ordering between graph snapshots, Naiad supports out-of-order execution which allows for new updates to be processed before the current computation finishes. Rather than grouping updates into *windows* to ensure correctness, Naiad opts for *versioning* vertex states, which are stored together with their logical timestamps. This is what the system makes use of to resolve conflicts when updates propagate through the graph.

Kappa is, similarly to Kineograph, designed for processing streaming graph data but, contrary to Kineograph, it is a single-machine in-memory system. The snapshotting scheme of Kappa aims to be more flexible by relaxing the barrier between updates and computations for the next batch of data. This is similar in spirit to Naiad, and would allow for finer-granularity processing without having to keep previous vertex states in the system.

Chapter 3

Design of Kappa

In this chapter, I will first provide a high-level description of the Kappa, an in-memory stream graph processing system designed to run on a single machine. Then, its update-centric computation, data and threading models will be explained in more detail.

3.1 Outline of the system

Contrary to many existing graph processing systems, Kappa does not first build a graph out of the input data and then run a query over it. Rather, given a query Q and an input stream S, Kappa will run Q over the inbound data coming from S.

The first building block of the system is, thus, the **ingestion engine**: raw data from the input stream is parsed and translated into *graph updates*. At the moment, the only two operations supported are edge additions and deletions, but the system could be easily extended to support vertex additions, deletions and modifications, and edge modifications for weighted graphs as well.

Graph updates model the activities happening in a real-world network. In a social platform, they would represent interactions between users, such as following or unfollowing friends, mentioning them or sharing their posts.

Once a sequence of updates has been applied to the graph, the **computation engine** takes care of rerunning the user-defined query on the portion of the graph affected by the updates, thus making the computation (1) localised and (2) incremental, as the previous vertex states are retained to compute the new states consistently with the new topology of the graph.

The number of updates in a sequence will be referred to as *batch size*. Finding its optimal value is still under investigation, and so is synchronising the ingestion and the computation engines to achieve a greater level of parallelism in the system. More detail on this latter issue is provided in Section 3.4.

To conclude this section, the general workflow of a stream graph processing system such as Kappa is shown in Figure 3.1: in this example, a popular social network is assumed to be the source of the data stream; interactions between users modify the input graph, on which a variety of different computations can be run continuously to gain near real-time analytics.

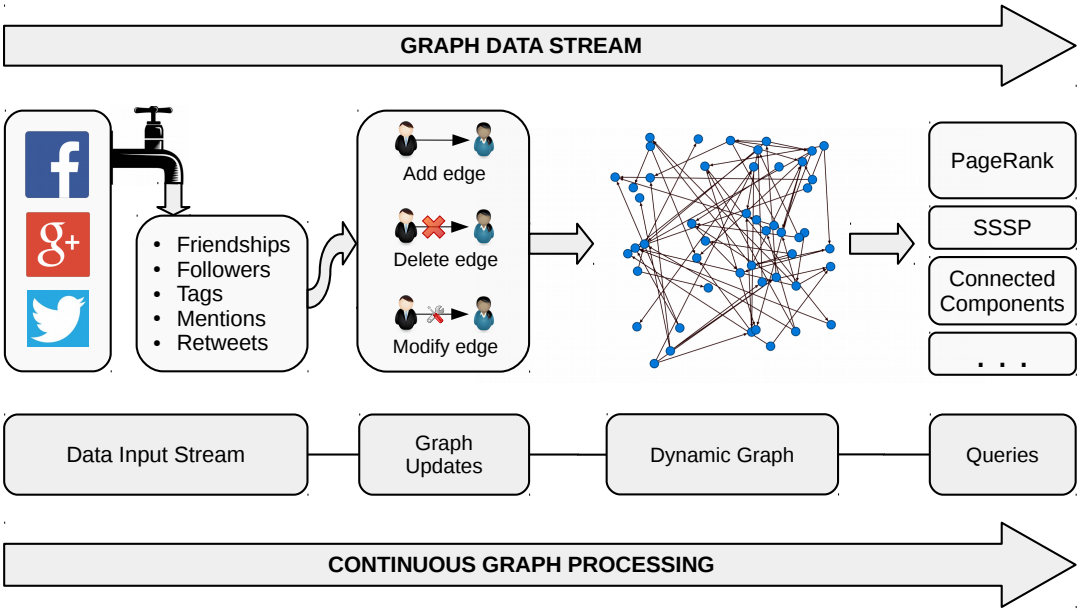

Figure 3.1: Workflow of a stream graph processing system.

3.2 Model of computation

Kappa takes as input a stream of graph updates, each of which modifies the graph by adding or deleting an edge. An edge is of the form (id_s, id_d), where id_s (resp. id_d) is the unique identifier of the source (resp. destination) vertex of the edge.

For each update, a user-defined function is invoked to act upon it. The update might or might not propagate through the graph, depending on the logic of the algorithm. Since the computation is triggered by graph updates, it is referred to as **update-centric**.

Listing 3.1: The programming interface of Kappa.

```
namespace UserDefinedComputation {
  void init_state (Digraph *g, vertex_id_t v);
  void on_activate(Digraph *g, vertex_id_t v);
  void on_add_edge(Digraph *g, vertex_id_t src,
                   vertex_id_t dst);
  void on_remove_edge(Digraph *g, vertex_id_t src,
                      vertex_id_t dst);
}
```

The programming model of Kappa shares similarities both with the **vertex-centric** and the **edge-centric** programming models described in Section 2.1, but it is fundamentally different from both of them. In fact, there is no notion of function being applied to *all* the vertices or edges at any given point during the computation. Rather, the computation proceeds in an event-driven fashion until an update—that is, a change in the graph topology which causes a number of vertex states to change, too—stops propagating.

The entry points of any computation are the `on_add_edge` and `on_remove_edge` functions, called by the system every time an edge is added or removed, respectively. The intuition behind the need for these two functions is that, for certain algorithms, an edge addition or deletion might not change the value of any vertex—in other words, it would not involve any computation to be carried out on the rest of the graph. As an example, one may think of computing the connected components of a given graph: adding an edge between two vertices which are already in the same component does not change the state of any vertex. On the contrary, if an edge is added between two previously disconnected vertices, then more work needs to be done. In such a case, the vertices affected by the update would be scheduled for execution, so that they can (1) recompute their states and (2) possibly schedule additional vertices for execution.

The logic for recomputing a vertex state can be expressed in the `on_activate` function, which is invoked by the system whenever a vertex is scheduled for execution. Typically, a vertex will pull the states of its neighbours, recalculate its own state and, provided that the latter has changed significantly, schedule its neighbours for execution.

Lastly, `init_state` is guaranteed to be called by the system only once to initialise the state of a newly added vertex. It should be noted that a vertex v is added to the graph the first time an update containing v comes from the input stream, but only if v is not already present.

Chapter 4 includes the implementation of some representative graph algorithms

using the interface above, which should help to understand how the system works and how to develop further algorithms.

3.3 Data layout

The data layout of Kappa has been designed to be as simple as possible, while still focusing on fast access and modification of graph data. To this aim, rather than creating several objects to represent different pieces of information, two main data structures have been used to store the vertex states and the graph topology. Both of them are kept in RAM, thus making Kappa an **in-memory** system.

Since the majority of real-world graphs are directed, only directed graphs are considered for the moment in Kappa. A single class `Digraph` contains all the information to store the input data. Once instantiated, this object allocates enough space to store (1) an array `states` holding all the vertex states and (2) an array `topology` representing the graph topology. Thus, the state of vertex i is stored in `states[i]`, whereas its in- and out-neighbours can be accessed through the structure `Dvertex` stored in `topology[i]`, as shown in Figure 3.2.

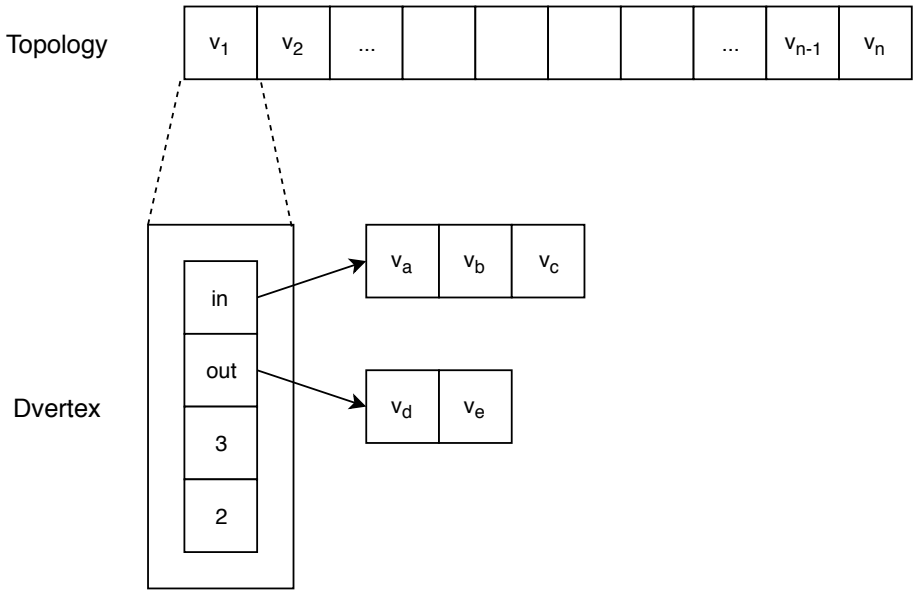

Figure 3.2: Data structures used to store the topology of the graph: `topology[i]` is an element of type `Dvertex`, which contains two pointers to the in- and out-neighbours, plus two incrementally maintained helper fields storing $degree_{in}(v_i)$ and $degree_{out}(v_i)$, respectively.

Since vertices can be added and removed in Kappa, an additional bitmap keeps

track of which vertices actually exist in the graph, as shown below in Figure 3.3.

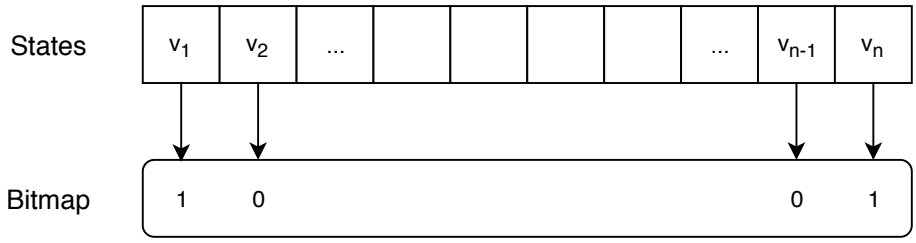

Figure 3.3: A bitmap keeps track of which vertices exist in the graph. In this example, v_1 and v_n are the IDs of valid vertices, whereas v_2 and v_{n-1} do not exist yet.

Finally, another design decision pertaining to memory usage was to avoid dynamic allocation of structures and objects at run time. To achieve this, the data structures needed for the various tasks performed by Kappa are instantiated through memory pools: by doing so, system calls to request memory dynamically are avoided, thus making the performance of the system faster.

3.4 Threading model

Kappa targets a **single-machine** environment, with a view to scale up by exploiting the parallelism offered by a modern multi-core machine as aggressively as possible. The argument of scale-up versus scale-out for graph processing is well explained by Lin in [7]: the author advocates to tackle the problem of graph processing starting with a scale-up solution first, and then scale out if needed. This not only allows for an easier implementation of the system, but also avoids the complexities brought by a distributed infrastructure on which the software has to be deployed and maintained.

At the time of writing this report, there are two different types of threads in the system: a main thread and a machine-dependent number of worker threads. Essentially, the main thread reads data from the input stream, converts it into tasks and inserts them into a queue. The worker threads pop tasks from the queue and execute them.

Tasks and the multi-producer, multi-consumer **queue** which holds them are the fundamental blocks of concurrency in the system. Tasks can be *update* or *computation* tasks: the former are used to update the graph topology, whereas the latter run the user-defined functions described in Section 3.2.

After a batch of updates of size b has been applied to the graphs, b computation tasks are enqueued into the queue to start the computation phase. Computation tasks spawn other computation tasks until the no new vertices are activated. Then, the next batch of updates is applied to the graph and the computation proceeds in the same way as just described. In this workflow, there is a **barrier** between the update and the computation phase, which limits the degree of parallelism whenever the number of tasks in the queue is less than the available number of threads. Softening this barrier—shown in Figure 3.4—is currently under investigation and a possible future direction for the project.

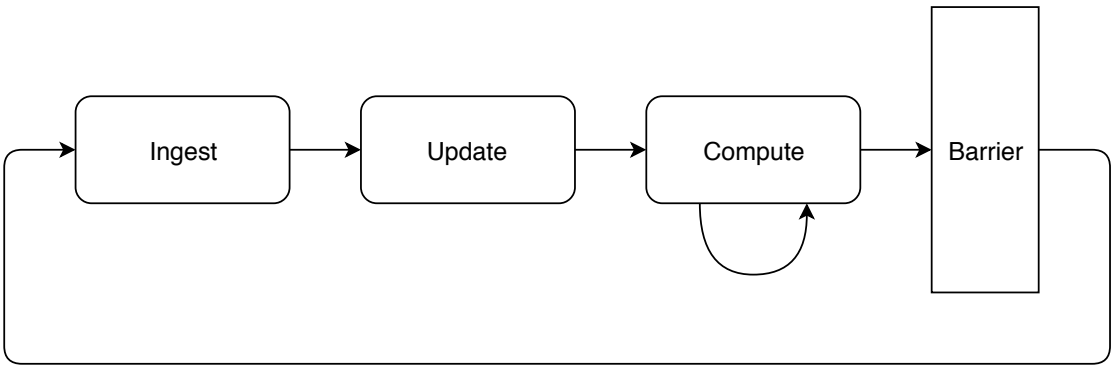

Figure 3.4: After ingesting a batch of updates, the system goes through the *update phase*, which is followed by the *computation phase*. The process is repeated until the system is stopped.

The correctness of the system is guaranteed by the tasks being executed in a FIFO order. In particular, this means that (1) all the graph updates are applied before the computation phase begins and (2) the changes caused by the updates propagate orderly through the portion of the graph affected by them.

To understand the ordering of events happening in the system, let us look at Figure 3.5. In this example, s_i represents a snapshot of the task queue at different times. During the update phase, an edge is added between v_1 and v_2—this operation is performed when the update task U_0 is popped from the queue and executed. After this, the system inserts into the queue a computation task C_1, which calls the user-defined function `on_add_edge` as a consequence of the edge (v_1, v_2) being added to the graph. C_1 activates v_3 and v_4, which results into two computation tasks C_{2a} and C_{2b} being pushed into the queue. Finally, v_4 activates v_5, whose state is recomputed when C_3 is popped from the queue. At this point, since all the computation tasks have finished their work and no other vertices have been activated, the system can proceed to the next update phase.

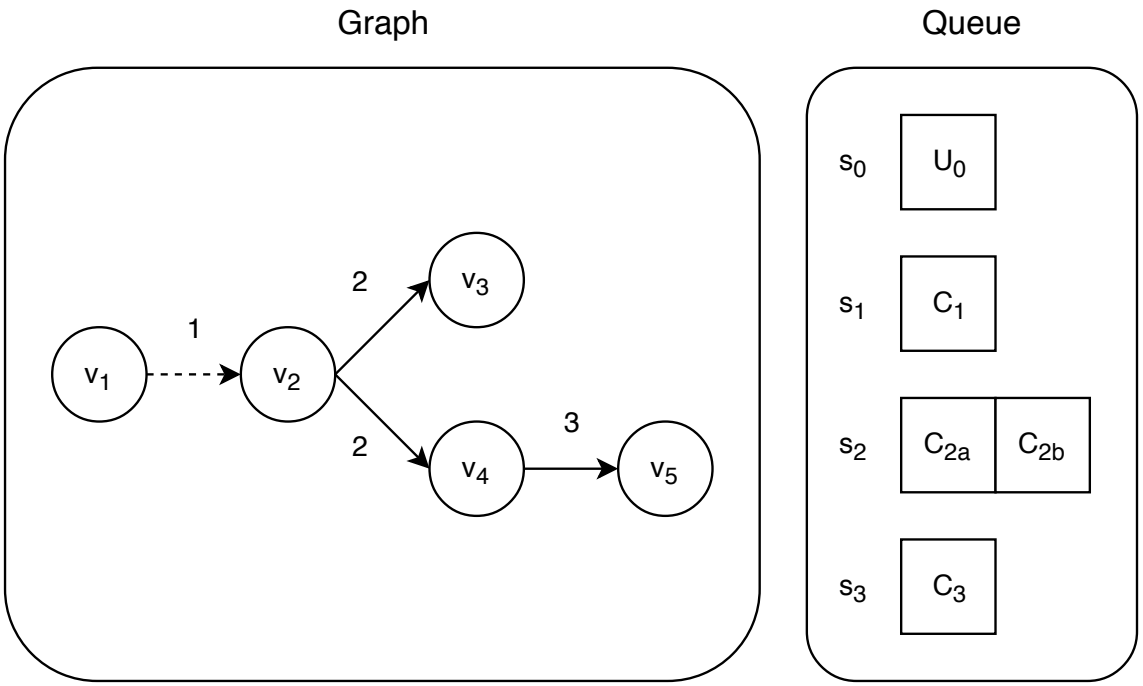

Figure 3.5: Four snapshots of the task queue as an update propagates through the graph.

3.4.1 Task compaction

The correctness of the system is based on the fact that a vertex v recomputes its value every time it is affected by an update propagating through the graph. In practice, this means that a computation task for v is pushed into the queue possibly several times after a batch of updates has been applied to the graph. Although, if a computation task C_v for a vertex v is already in the queue when v gets activated by some other vertex, there is no need to add another computation task for v to the queue: indeed, since v is scheduled, its state will be recomputed during the execution of C_v.

This optimisation is referred to as **task compaction**, since several different tasks for the same vertex are logically merged together into one. An example of this is shown in Figure 3.6.

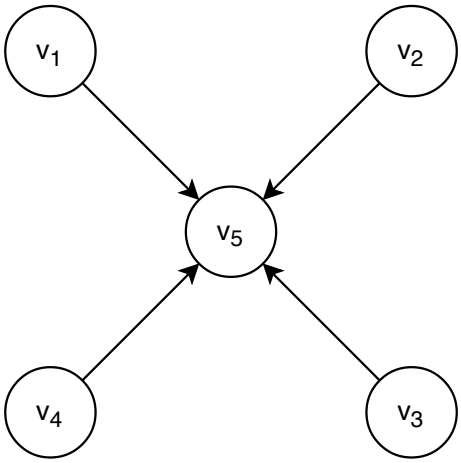

Figure 3.6: When recomputing their states, v_1, v_2, v_3 and v_4 activate vertex v_5. Although, only one computation task is pushed into the queue for v_5 instead of four.

Task compaction is beneficial especially when dealing with real-world graphs exhibiting a skewed degree distribution. This is quite common in social networks, where the degree of the vertices representing celebrities is much larger than that of regular users.

Chapter 4

Implementation

This chapter presents three algorithms developed to run on Kappa and provides more detail on how threads of execution are handled in the system.

4.1 PageRank

PageRank (**PR**) is an algorithm originally developed to rank web pages by their importance. The main idea relies around the number and the quality of incoming links a web page has: the higher this number, the more important a web page is.

A PageRank algorithm runs for a fixed number of iterations or until convergence is reached. In each iteration, every vertex v spreads its current value divided by $degree_{out}(v)$ across its outgoing edges and recomputes its state by summing up the values received from its in-neighbours. The pseudocode to express this algorithm in a Pregel-like system is given below.

Listing 4.1: Pseudocode to express a PageRank algorithm running for 30 iterations in a Pregel-like system.

```
1 class PageRankVertex : public Vertex {
2   void compute(MessageIterator *msgs) {
3     double sum = 0;
4
5     for (; !msgs->done(); msgs->next())
6       sum += msgs->value();
7
8     this->value = 0.15 / no_of_vertices() + 0.85 * sum;
9
```

```
10      if (iteration() < 30) {
11        send_to_neighbours(this->value / this->out_degree);
12      }
13    }
14  }
```

Let us see how to express this algorithm in Kappa. We begin by defining what happens when an edge (src, dst) is added or removed: since $degree_{out}(src)$ has increased or decreased by 1 after the update has been applied between t_i and t_{i+1}, all the neighbours of src at time t_i and t_{i+1} need to recompute their states.

Listing 4.2: Definition of `on_add_edge` and `on_remove_edge` to compute PageRank in Kappa.

```
1  void on_add_edge(Digraph *g, vertex_id_t src,
2                   vertex_id_t dst) {
3    for (auto neighbour : *(g->get_out_neighborhood(src))) {
4      g->activate_vertex(neighbour);
5    }
6  }
7
8  void on_remove_edge(Digraph *g, vertex_id_t src,
9                      vertex_id_t dst) {
10   for (auto neighbour : *(g->get_out_neighborhood(src))) {
11     g->activate_vertex(neighbour);
12   }
13
14   g->activate_vertex(dst);
15 }
```

Then, we proceed by defining the behaviour of a vertex when it gets activated. We can do so in `on_activate`, which resembles the `compute` function shown in Listing 4.1. Although, since Kappa does not use message passing as a communication method between vertices, a vertex can directly read its neighbours' values to compute its new PageRank.

Listing 4.3: Definition of `on_activate` to compute PageRank in Kappa.

```
1  void on_activate(Digraph *g, vertex_id_t v) {
2    double sum = 0.0;
3
4    for (auto neighbour : *(g->get_in_neighborhood(v))) {
5      graph_size_t out_degree = g->get_out_degree(neighbour);
6
```

```
7       if (out_degree > 0) {
8         sum += g->get_state(neighbour) / out_degree;
9       }
10    }
11
12    double new_pr = 0.15 / g->get_order() + 0.85 * sum;
13
14    double old_pr = g->get_state(v);
15    g->set_state(v, new_pr);
16
17    if (std::abs(new_pr - old_pr) >= EPSILON) {
18      for (auto neighbour : *(g->get_out_neighborhood(v))) {
19        g->activate_vertex(neighbour);
20      }
21    }
22  }
```

In addition to encapsulating the logic for state computation, `on_activate` also contains the termination condition for a given algorithm. For example, at lines 17–21 in the code above, a vertex will not schedule its out-neighbours for execution unless its value has changed by more than a constant `EPSILON`. Effectively, this means that an update will stop propagating at this point, even though a vertex which has already recomputed its state once during the current iteration might still be activated by its neighbours during the same iteration. This scenario depends on the topology of the graph and the nature of the algorithm itself.

4.2 Single-source shortest paths

A **single-source shortest paths** (**SSSP**) algorithm finds all the minimal paths between a given source vertex and every other vertex. One of the many applications of this algorithm is to compute the shortest route between two points in a geographical map, which is clearly of great practical interest.

To compute the shortest paths in Kappa, let us start by defining `init_state` this time.

Listing 4.4: Definition of `init_state` to compute SSSP in Kappa.

```
1  void init_state(Digraph *g, vertex_id_t v) {
2    state_t state =
3      (v == SOURCE) ? 0 : get_min_distance(g, v) + 1;
```

```
4
5     g->set_state(v, state);
6   }
```

In the code above, `SOURCE` is the source vertex, whereas `get_min_distance` is a helper function which returns the minimum state stored in the neighbours of a vertex. In this case, the state of a vertex v corresponds to the distance from the source vertex s in terms of the minimum number of hops to reach v from s.

Now, let us see what happens when an update comes in from the input stream.

Listing 4.5: Definition of `on_add_edge` and `on_remove_edge` to compute SSSP in Kappa.

```
1  void on_add_edge(Digraph *g, vertex_id_t src,
2                   vertex_id_t dst) {
3    state_t src_state = g->get_state(src);
4    state_t dst_state = g->get_state(dst);
5
6    if (src_state + 1 < dst_state) {
7      g->set_state(dst, src_state + 1);
8
9      for (auto neighbour : *(g->get_out_neighborhood(dst))) {
10       g->activate_vertex(neighbour);
11     }
12   }
13 }
14
15 void on_remove_edge(Digraph *g, vertex_id_t src,
16                     vertex_id_t dst) {
17   if (g->get_state(src) + 1 == g->get_state(dst)) {
18     g->activate_vertex(dst);
19   }
20 }
```

These functions are pretty similar in spirit: they both check whether an edge addition or deletion will cause other vertices to update their states. As an example, let us consider two vertices src and dst, and let us assume that $state(src) + 1 > state(dst)$. Once an edge is added between the two, dst can be reached passing through src in $state(src) + 1$ hops. However, due to our assumptions, $state(src) + 1 > state(dst)$, which means that the edge (src, dst) will not really change dst's state, and thus will not propagate through the graph. On the contrary, if $state(src) + 1 < state(dst)$, then a new edge (src, dst) will create a new,

shorter path between the source vertex and dst—in this case, dst's out-neighbours will be scheduled for execution.

Finally, `on_activate` will be invoked by the system to recompute the state of every scheduled vertex.

Listing 4.6: Definition of `on_activate` to compute SSSP in Kappa.

```
1  void on_activate(Digraph *g, vertex_id_t v) {
2    state_t old_distance = g->get_state(v);
3    state_t min = get_min_distance(g, v);
4
5    if (old_distance > min + 1) {
6      g->set_state(v, min + 1);
7
8      for (auto neighbour : *(g->get_out_neighborhood(v))) {
9        g->activate_vertex(neighbour);
10     }
11   }
12 }
```

Even though the algorithm explained above does not consider weighted graphs, it can be easily extended to take them into account. Indeed, this could be achieved by associating each edge with a numeric value representing its weight, rather than assuming each edge to have a traversal cost of 1.

4.3 Weakly connected components

A subset of vertices V of a directed graph is considered *weakly connected* if, for every pair of vertices $(u, v) \in V \times V$, there is an undirected path between u and v. In other words, a path can be formed by traversing the edges regardless of their orientation.

Computing the **weakly connected components** of a directed graph means finding all the maximal subsets of vertices $V_i \subseteq G$ such that each pair (u, v) in V_i is weakly connected and, for each pair (u, v) where $u \in V_i$ and $v \in V_j$ with $i \neq j$, there is no undirected path between u and v. An intuitive method to do so is what is known as *label propagation*: each vertex sets its state to some label which identifies a component, and then propagates it to its neighbours. At the end of the algorithm, each component is uniquely identified by the vertices sharing the same label.

To reason about how to write such an algorithm on a dynamically changing graph, let us imagine a graph whose weakly connected components have been identified

already. Now, an edge addition might either (1) connect two previously disconnected components or (2) connect two vertices already in same component. If the latter case happens more often then the former, an edge addition will not trigger significant work to be carried out on the rest of the graph. Interestingly, if we consider the popular `twitter-2010` [4] as input graph, around 80% of its vertices are in the same component: therefore, if the update stream at hand was to add a large number of edges in this component, the computation engine would have little work to do.

On the other hand, an edge removal is not so easy to reason about. In fact, one would need to detect whether a deleted edge breaks the connectivity between two components, as shown in Figure 4.1.

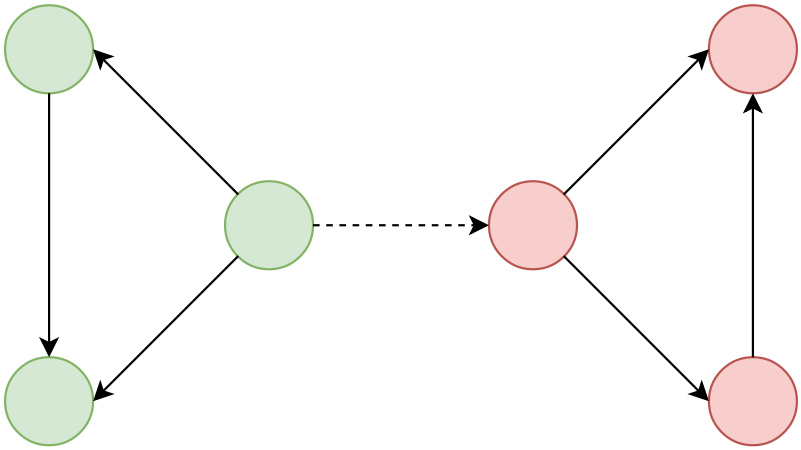

Figure 4.1: The graph is split into two components when the dashed edge is deleted.

Again, let us see how `on_add_edge` and `on_remove_edge` are implemented.

Listing 4.7: Definition of `on_add_edge` to compute weakly connected components in Kappa.

```
1  void on_add_edge(Digraph *g, vertex_id_t src,
2                   vertex_id_t dst) {
3    state_t src_state = g->get_state(src);
4    state_t dst_state = g->get_state(dst);
5
6    if (src_state != dst_state) {
7      vertex_id_t smaller_v =
8        (src_state < dst_state) ? src : dst;
9
10     state_t new_label =
11       (src_state < dst_state) ? dst_state : src_state;
12
```

```
13      g->set_state(smaller_v, new_label);
14
15      for (auto neighbour :
16           g->get_in_out_neighborhood(smaller_v)) {
17        g->activate_vertex(neighbour);
18      }
19    }
20  }
```

In `on_add_edge`, we first check whether `src` and `dst` already belong to the same component by comparing the labels stored in their states. If this is not the case, we choose the greater label `new_label` between the two, set the vertex with the smaller label to `new_label` and finally propagate the new label through its component.

Listing 4.8: Definition of `on_remove_edge` to compute weakly connected components in Kappa.

```
1  void on_remove_edge(Digraph *g, vertex_id_t src,
2                      vertex_id_t dst) {
3    g->set_state(src, get_new_label());
4
5    for (auto neighbour : g->get_in_out_neighborhood(src)) {
6      g->activate_vertex(neighbour);
7    }
8
9    g->set_state(dst, get_new_label());
10
11   for (auto neighbour : g->get_in_out_neighborhood(dst)) {
12     g->activate_vertex(neighbour);
13   }
14 }
```

As mentioned above, `on_remove_edge` has to deal with the case of an edge removal splitting a component into two. One approach would be to check whether `dst` is still reachable from `src`: in this scenario, no additional work needs to be done. Although, this would come at the cost of stopping the computation to start a visit from `src` to detect if `dst` was still reachable. Alternatively, a simpler implementation is based on the idea of assuming that an edge removal always breaks the connectivity between `src` and `dst`. This latter approach brings the benefit of an easier logic to implement, and seems to perform better in practice. Therefore, the code above sets both the states of `src` and `dst` to two new different labels, and then propagates them through the graph. If connectivity had not been broken in

the first place, the result of this operation will still guarantee correctness—namely, `src` and `dst` will end up sharing the same label, whichever is bigger between the two newly generated ones.

For completeness, `get_new_label` is a helper function which returns a unique, monotonically increasingly label every time it is called.

In `on_activate`, a vertex pulls its neighbours' states, sets its own to the largest amongst the vertices in its neighbourhood and propagate its new label if different from its previous one.

Listing 4.9: Definition of `on_activate` to compute weakly connected components in Kappa.

```
1  void on_activate(Digraph *g, vertex_id_t v) {
2    state_t old_label = g->get_state(v);
3
4    state_t max_label = get_max_label(g, v);
5    g->set_state(v, max_label);
6
7    if (old_label != max_label) {
8      for (auto neighbour : g->get_in_out_neighborhood(v)) {
9        g->activate_vertex(neighbour);
10     }
11   }
12 }
```

Finally, `init_state` sets the initial value of a vertex v to the label of the component to which v belongs at the time when it is added to the graph.

Listing 4.10: Definition of `init_state` to compute weakly connected components in Kappa.

```
1  void init_state(Digraph *g, vertex_id_t v) {
2    g->set_state(v, get_max_label(g, v));
3  }
```

4.4 Handling threads

Kappa aims to scale up by fully utilising the resources offered by a single machine. To achieve this, each physical core should be entirely loaded most of the time to speed up the computation. The logic to create and handle threads for each core made available to the system is encapsulated in the `ThreadPool` class.

As described in Section 3.4, tasks are the units of computation in the system. A task is pushed into a concurrent queue, and it is executed once a worker thread removes it from the queue. The `ThreadPool` class not only provides the interface to submit a new task, but also creates a set of threads at start-up which are reused by the system. This avoids the overhead of constructing a thread every time a task has to be executed.

Listing 4.11: Interface of the `ThreadPool` class.

```
class ThreadPool {
public:
  ThreadPool();
  ~ThreadPool();

  void init_threads(const uint no_of_threads,
                    const uint offset = 0);
  void init_numa_node(const uint node_no,
                      const uint no_of_threads = 16);
  void init_numa_nodes(const std::vector<uint>);

  void submit(Task*);
  void barrier();

private:
  void worker(void);
  void destroy(void);

  std::vector<std::thread> threads;
  ThreadSafeQueue<Task*> task_queue;
  std::atomic_ushort active;
  std::atomic_bool done;
};
```

Listing 4.11 shows the interface offered by the `ThreadPool` class. Once a task is submitted to the thread pool via the `submit` function, it is guaranteed to be executed by one of its threads. The logic to implement a barrier between the computation phase and the next update phase is encapsulated in the `barrier` function. A thread calling this function will stop until (1) there are no tasks pending in the queue and (2) all the threads allocated by the thread pool are idle. The second condition ensures that, even when the queue is empty, no new tasks will be spawn by a currently executing task.

Furthermore, our thread pool offers three functions to initialise its set of threads:

`init_threads`, `init_numa_node` and `init_numa_nodes`. These are described in the next section.

4.4.1 Thread placement

Nowadays, it is very common for multi-core machines to have a NUMA architecture. Without going into detail, a NUMA system is comprised of different NUMA nodes: each of these can be thought of as a set of CPUs and some amount of local memory. Similarly to a distributed system, a NUMA node can access non-local memory on another node, but this operation is more expensive than accessing its own memory.

For this reason, the thread pool described above supports **NUMA-aware thread placement**. To understand how this works, let us assume that our machine has 4 NUMA nodes, each consisting of 16 physical CPUs, as summarised in the table below.

NUMA node	CPU IDs
1	0, 4, 8, . . . , 60
2	1, 5, 9, . . . , 61
3	2, 6, 10, . . . , 62
4	3, 7, 11, . . . , 63

Table 4.1: Each NUMA node has 16 CPUs, whose IDs are shown in the right-hand column.

Now, calling `init_numa_node(1)` will create 16 threads and pin them to CPUs 0, 4, 8, . . . , 60. This will prevent threads migrating from one NUMA node to another. Alternatively, it is possible to specify the number of threads to be allocated on a NUMA node by passing the optional parameter `no_of_threads` to `init_numa_node`. A similar logic is implemented in `init_numa_nodes` to utilise multiple NUMA nodes at the same time.

Finally, our thread pool can also create a given number of threads and pin them sequentially via the function `init_threads`.

Chapter 5

Evaluation

This chapter explains how Kappa has been evaluated by first describing the environment and the data used for the experiments, and then by reasoning about the results obtained thus far. The system has been tested for correctness by comparing its output—namely, the vertex states—with that produced by another graph processing system. To this aim, we have set up several Python scripts that run the same algorithms implemented in Kappa using the popular graph library NetworkX.

5.1 Testbed

The system used for our tests is a single NUMA machine running Ubuntu 16.04 equipped with four Intel Xeon E5-4660 16-core processors (which add up to 64 cores in total) and 512GB of RAM. Our machine supports hyper-threading, which means that each physical core is treated as two virtual cores by the operating system. Although, since our system is computation-bound and not I/O-bound, sharing a core between two threads will not lead to any performance benefit since the two threads will be interfering with each other. For this reason, we have made use of physical cores only in our experiments.

Kappa has been implemented in C++ and compiled with `gcc`. Furthermore, as mentioned above, some Python code has been used to test the system for correctness.

5.2 Data source

Rather than relying on available temporal datasets, whose size is not large enough to carry out significant experiments, we have opted for simulating a stream of temporal updates by using a static dataset.

To this purpose, we have used the Twitter-based Higgs dataset [6] as follows: for each test, a core graph is built using 14 million edges from the dataset; then, each vertex is initialised to a precomputed value which is algorithm-dependent; finally, the remaining edges are sent to the system in batches to simulate a stream of updates.

In total, the Higgs dataset has 456,626 vertices and 14,855,842 edges.

5.3 Experiments

This section contains the experiments conducted so far to measure the performance and scalability of the system when running PageRank and single-source shortest paths. These two algorithms present different workloads: PageRank tends to take longer to converge, and the tasks it generates are heavier compared to those generated by SSSP.

5.3.1 SSSP

The following plots show the time taken for five computation phases to complete after five update phases have finished. Each update phase applies a batch of 100,000 updates to the graph. Furthermore, the reported computation time is the average of five sequential runs.

Figure 5.1 shows how the system speeds up when given more cores. However, this trend stops at 8 cores, after which the computation time actually increases. So, we decided to reduce the number of system calls, especially for requesting memory dynamically: since tasks are short-lived objects created very often, a memory pool was introduced in the system to handle them.

In Figure 5.2, we can notice a drop at 16 cores. This is due not only to the task pool mentioned above, but also to task compaction—this optimisation is described in Section 3.4.1.

At this point, after analysing our system with some profiling tools, we noticed that significant time was spent locking and unlocking the task queue. Therefore, we

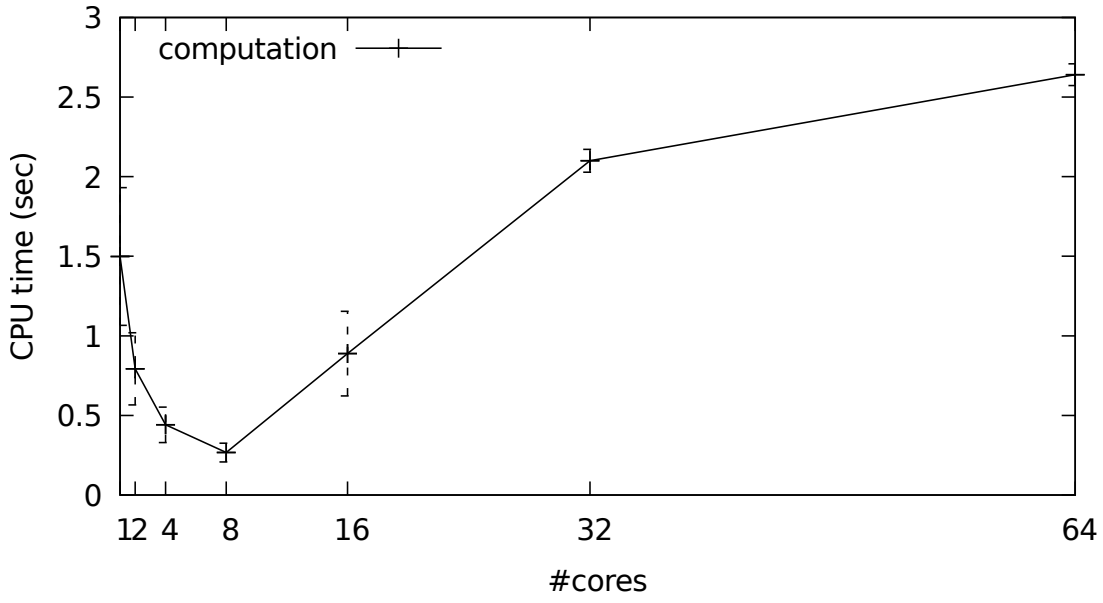

Figure 5.1: Initial results for SSSP.

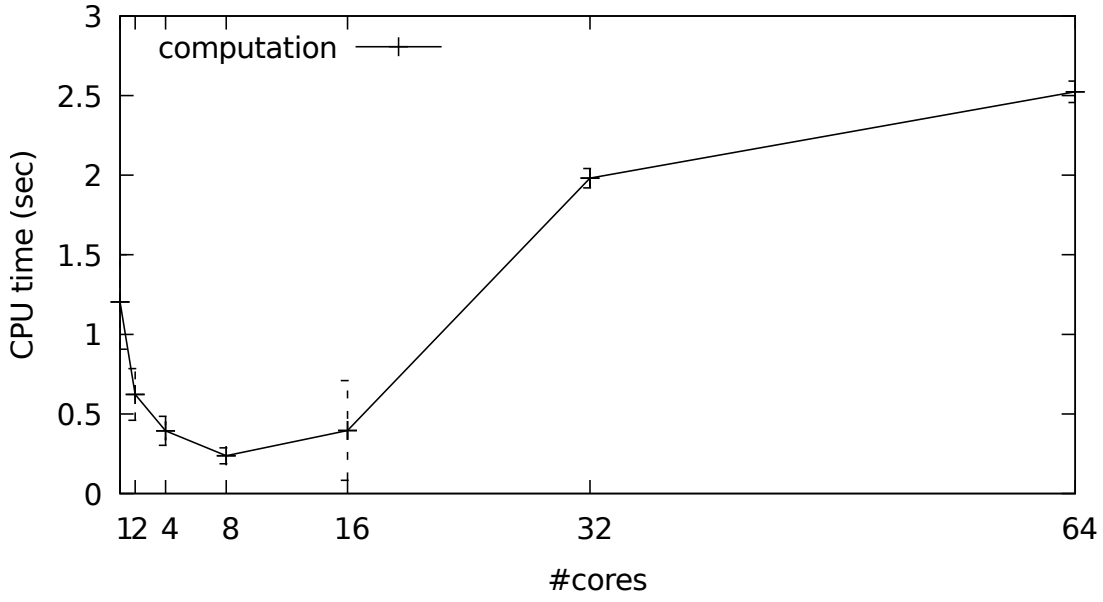

Figure 5.2: Performance of SSSP with task pool and task compaction.

replaced it with a lock-free queue, which resulted in both better performance and scalability.

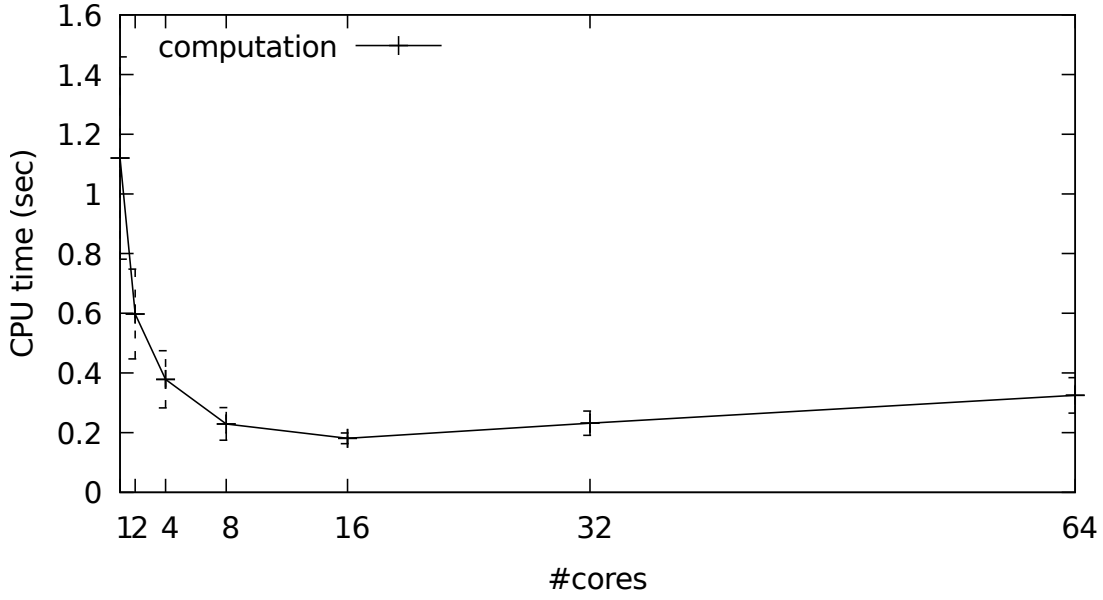

Figure 5.3: Performance of SSSP after the addition of a lock-free queue.

The plot in Figure 5.3 demonstrates not only how the system scales better up to 16 cores, but also how the computation time is overall reduced.

Achieving better scalability when using more than 16 cores is under investigation, but we suspect this behaviour is due to (1) cores on different NUMA nodes accessing remote memory, (2) not generating enough work for all the available cores and (3) not having implemented NUMA-aware data placement in different memory regions.

5.3.2 PageRank

The experiments for PageRank have been conducted in the same way outlined above. It is reported in this section how the algorithm performs after the addition of the optimisations previously described and a lock-free queue.

In Figure 5.4, we observe better scalability, but significantly slower computation time compared to SSSP. This is due to the heavier computation tasks generated by PageRank—in this case, the queue is not the bottleneck.

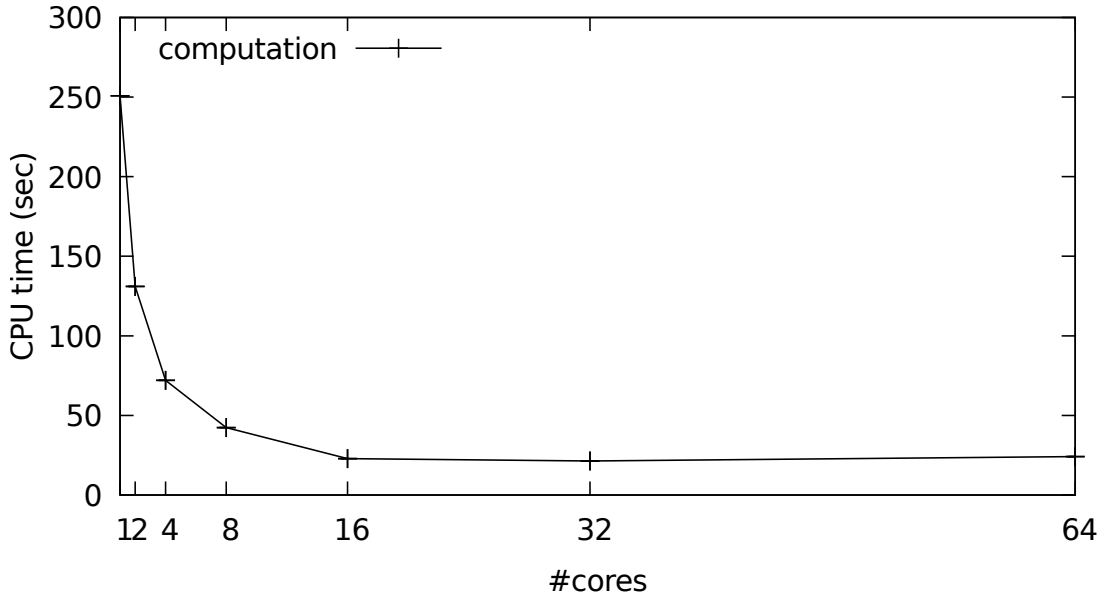

Figure 5.4: Computation time for running PageRank using an increasing number of cores.

Improving the performance of PageRank, in particular with the aim to lower the computation time when using a single thread, is currently under investigation.

Chapter 6

Conclusion

In my thesis, I have described the current state of Kappa and its programming model, which represents my main contribution to it. Although, this report depicts just a snapshot of the system at the time of writing. In fact, Kappa is expected to evolve in the coming weeks.

Future work is going to be focused on softening the barrier between the computation phase and the update phase. This might be achieved by timestamping the system tasks to detect whether it is possible to start the computation on a part of the graph that lives in t_i while other parts are still in t_{i-1}.

A notion of *stability* might be introduced in the system: a stable vertex at time t_i is a vertex whose state will not change any more until t_{i+1}. If a mechanism to identify stable vertices was in place, then the computation for such vertices could proceed without having to wait for the previous phase to complete, and it would still guarantee correct results.

Furthermore, it might be beneficial to divide the graph data into different memory regions belonging to different NUMA nodes. This might lower the computation time by significantly reducing expensive remote memory accesses. To this purpose, our task queue might also be split into several queues, one for each NUMA node.

Minor additions to the system include making it possible for the user to define the type of the value stored in a vertex and offering support for undirected and weighted graphs.

Appendix A

Legal and ethical considerations

	Yes	No
Section 1: HUMAN EMBRYOS/FOETUSES		
Does your project involve Human Embryonic Stem Cells?		✓
Does your project involve the use of human embryos?		✓
Does your project involve the use of human foetal tissues / cells?		✓
Section 2: HUMANS		
Does your project involve human participants?		✓
Section 3: HUMAN CELLS / TISSUES		
Does your project involve human cells or tissues? (Other than from Human Embryos/Foetuses i.e. Section 1)?		✓
Section 4: PROTECTION OF PERSONAL DATA		
Does your project involve personal data collection and/or processing?		✓
Does it involve the collection and/or processing of sensitive personal data (e.g. health, sexual lifestyle, ethnicity, political opinion, religious or philosophical conviction)?		✓
Does it involve processing of genetic information?		✓
Does it involve tracking or observation of participants? It should be noted that this issue is not limited to surveillance or localization data. It also applies to Wan data such as IP address, MACs, cookies etc.		✓
Does your project involve further processing of previously collected personal data (secondary use)? For example Does your project involve merging existing data sets?		✓

Section 5: ANIMALS		
Does your project involve animals?		✓
Section 6: DEVELOPING COUNTRIES		
Does your project involve developing countries?		✓
If your project involves low and/or lower-middle income countries, are any benefit-sharing actions planned?		✓
Could the situation in the country put the individuals taking part in the project at risk?		✓
Section 7: ENVIRONMENTAL PROTECTION AND SAFETY		
Does your project involve the use of elements that may cause harm to the environment, animals or plants?		✓
Does your project deal with endangered fauna and/or flora /protected areas?		✓
Does your project involve the use of elements that may cause harm to humans, including project staff?		✓
Does your project involve other harmful materials or equipment, e.g. high-powered laser systems?		✓
Section 8: DUAL USE		
Does your project have the potential for military applications?		✓
Does your project have an exclusive civilian application focus?		✓
Will your project use or produce goods or information that will require export licenses in accordance with legislation on dual use items?		✓
Does your project affect current standards in military ethics e.g., global ban on weapons of mass destruction, issues of proportionality, discrimination of combatants and accountability in drone and autonomous robotics developments, incendiary or laser weapons?		✓
Section 9: MISUSE		
Does your project have the potential for malevolent/criminal/terrorist abuse?		✓
Does your project involve information on/or the use of biological-, chemical-, nuclear/radiological-security sensitive materials and explosives, and means of their delivery?		✓

Does your project involve the development of technologies or the creation of information that could have severe negative impacts on human rights standards (e.g. privacy, stigmatization, discrimination), if misapplied?		✓
Does your project have the potential for terrorist or criminal abuse e.g. infrastructural vulnerability studies, cybersecurity related project?		✓
Section 10: LEGAL ISSUES		
Will your project use or produce software for which there are copyright licensing implications?		✓
Will your project use or produce goods or information for which there are data protection, or other legal implications?		✓
Section 11: OTHER ETHICS ISSUES		
Are there any other ethics issues that should be taken into consideration?		✓

Neither Kappa nor my contributions to it depend on any third-party data or external interactions. Building a system, per se, is often a self-contained activity.

The datasets mentioned throughout this report which have been used to run the experiments are all publicly available. Furthermore, they have all been anonymised before being released.

Bibliography

[1] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C Hsieh, Deborah A Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E Gruber. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, 26(2):4, 2008. pages 6

[2] Raymond Cheng, Ji Hong, Aapo Kyrola, Youshan Miao, Xuetian Weng, Ming Wu, Fan Yang, Lidong Zhou, Feng Zhao, and Enhong Chen. Kineograph: taking the pulse of a fast-changing and connected world. In *Proceedings of the 7th ACM european conference on Computer Systems*, pages 85–98. ACM, 2012. pages 11

[3] Wentao Han, Youshan Miao, Kaiwei Li, Ming Wu, Fan Yang, Lidong Zhou, Vijayan Prabhakaran, Wenguang Chen, and Enhong Chen. Chronos: a graph engine for temporal graph analysis. In *Proceedings of the Ninth European Conference on Computer Systems*, page 1. ACM, 2014. pages 11

[4] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. What is twitter, a social network or a news media? In *Proceedings of the 19th international conference on World wide web*, pages 591–600. ACM, 2010. pages 26

[5] Aapo Kyrola, Guy E Blelloch, and Carlos Guestrin. Graphchi: Large-scale graph computation on just a pc. USENIX, 2012. pages 10

[6] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. `http://snap.stanford.edu/data`, June 2014. pages 32

[7] J. Lin. Scale up or scale out for graph processing? *IEEE Internet Computing*, 22(3):72–78, 2018. pages 17

[8] Yucheng Low, Danny Bickson, Joseph Gonzalez, Carlos Guestrin, Aapo Kyrola, and Joseph M Hellerstein. Distributed graphlab: a framework for machine learning and data mining in the cloud. *Proceedings of the VLDB Endowment*, 5(8):716–727, 2012. pages 7, 9

[9] Grzegorz Malewicz, Matthew H Austern, Aart JC Bik, James C Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 135–146. ACM, 2010. pages 1, 2, 3, 6, 7, 8

[10] Derek G Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. Naiad: a timely dataflow system. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 439–455. ACM, 2013. pages 12

[11] Amitabha Roy, Laurent Bindschaedler, Jasmina Malicevic, and Willy Zwaenepoel. Chaos: Scale-out graph processing from secondary storage. In *Proceedings of the 25th Symposium on Operating Systems Principles*, pages 410–424. ACM, 2015. pages 10, 11

[12] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. X-stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 472–488. ACM, 2013. pages 3, 4, 8, 10

[13] Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *ACM Sigplan Notices*, volume 48, pages 135–146. ACM, 2013. pages 5

[14] Talkwalker, 2017. pages 1

[15] Da Yan, James Cheng, Yi Lu, and Wilfred Ng. Effective techniques for message reduction and load balancing in distributed graph computation. In *Proceedings of the 24th International Conference on World Wide Web*, pages 1307–1317. International World Wide Web Conferences Steering Committee, 2015. pages 7